

Ölü Bant (Deadband) Algoritması - Proje Raporu

1. Giriş

Ölü bant (deadband), kontrol sistemlerinde küçük hataların yok sayılması için kullanılan bir tekniktir. Hareketli uydu terminali projesinde, azimuth ve elevasyon motorlarının çok küçük açı hatalarında sürekli titremesini (hunting) önlemek için kullanılır. Bu sayede motor sürücüleri gereksiz yere yorulmaz, güç tüketimi azalır ve mekanik asınma minimuma iner.

2. Projede Kullanılan Ölü Bant Algoritmaları

2.1 Gömülü C++ PID Kontrolcü (PIDController.h)

Dosya: kerim unis/BTK_Satellite_Terminal/include/PIDController.h

PID sınıfının compute() metodunda ölü bant şu şekilde uygulanır:

```
float error = target - measured;
if (p_wrap_angle) {
    while (error > 180.0f) error -= 360.0f;
    while (error < -180.0f) error += 360.0f;
}
if (fabsf(error) < p_deadband) {
    state_integral = 0.0f;
    state_prev_error = 0.0f;
    state_output = 0.0f;
    return 0.0f;
}
```

Algoritma adımları:

1. Hedef ile ölçüm arasındaki fark (hata) hesaplanır
2. Azimuth eksenini için açı sarmalama uygulanır (-180 ile +180 derece arası)
3. Hatanın mutlak değeri ölü bant eşliğinden (config.h: PID_DEADBAND_DEG = 0.1 derece) küçükse:
 - Integral terimi sıfırlanır (anti-windup)
 - Önceki hata sıfırlanır (türev hesaplaması için)
 - Çıkış sıfıra set edilir
 - Motor komutu üretilmez (0 döner)
4. Hata eşikini aşarsa normal PID hesaplaması devam eder

Bu yaklaşım sayesinde:

- Anten hedef açısına ulaştığında (+/-0.1 derece hassasiyetle) motor durur
- Integral sıfırlandığı için birikmiş hata kalmaz
- Türev terimi sıfırlandığı için ani geri dönüşlerde spike oluşmaz

2.2 AntennaController (PIDController.h)

Çift eksenli kontrol yöneticisi olan AntennaController::update_internal() metodunda hedefte olma kontrolü:

```
out.az_on_target = (fabsf(out.az_error_deg) < PID_DEADBAND_DEG * 2.0f);
out.el_on_target = (fabsf(out.el_error_deg) < PID_DEADBAND_DEG * 2.0f);
```

Hedefte olma esigi, ölü bant esiginin 2 kati olarak belirlenmistir (0.2 derece). Bu, sistemin hedefe kilittendi bilgisini telemetriye göndermesi için kullanilir.

2.3 C++ Dataset Simulator (mergen_core.hpp, PidController)

Dosya: Efe/cpp_core/include/mergen_core.hpp

Burada ölü bant dolayli olarak RateLimitedAxis sinifinda backlash mekanizmasi ile uygulanir:

```
if (std::abs(error) <= backlash_deg_) return position_deg_;
```

Backlash (0.08 derece azimuth, 0.05 derece elevasyon) küçük hatalarin motor komutuna dönüşmesini engeller ve mekanik bosluklari taklit eder.

2.4 Python Dataset Simulator (dataset_simulator.py - eski versiyon)

Python tabanlı simulator Pid sinifinda ölü bant bulunmamaktadır. PID çıktisi sadece minimum ve maximum degerler arasinda clamp edilir.

2.5 C++ Stabilizasyon Örneği (x.cpp)

Dosya: simge_elif/x.cpp

MergenStabilizer sinifinda ölü bant bulunmamaktadır ancak integral windup korumasi vardır:

```
integral = std::clamp(integral + error * dt, -INTEGRAL_LIMIT, INTEGRAL_LIMIT);
```

Bu örnek eğitim amaçlıdır ve gerçek sistemde ölü bant eklenmesi önerilir.

3. Proje Spesifik Ölü Bant Parametreleri

Parametre	Deger	Açıklama
PID_DEADBAND_DEG	0.1 derece	Motor durdurma esigi
Azimuth hedefte olma	0.2 derece	PID_DEADBAND_DEG x 2
Elevasyon hedefte olma	0.2 derece	PID_DEADBAND_DEG x 2
Azimuth backlash	0.08 derece	Mekanik bosluk simülasyonu
Elevasyon backlash	0.05 derece	Mekanik bosluk simülasyonu

4. Ölü Bant Uygulamasının Faydaları

4.1 Mekanik Avantajlar

- Motor titresimi (hunting) önlenir: +/- 0.1 derece içinde motor sürekli ileri-geri gitmez
- PWM anahtarlama sayısı azalir, MOSFET/sürücü ömrü uzar
- Redüktör ve disli asinmasi minimuma iner
- Kablo sarmalama mekanizmasi gereksiz yorulmaz

4.2 Elektriksel Avantajlar

- Güç tüketimi azalir (motor beklemedeyken çekilen akim düşer)
- Elektromanyetik girişim (EMI) seviyesi düşer
- Pil ile çalışma durumunda enerji tasarrufu sağlanır

4.3 Sistem Performansı

- PID integral terimi birikmez, windup önlenir
- Hedef değistiginde hızlı yanıt verilir (integral sıfır)
- Türev terimi sıfırlandığı için gürültü amplifikasyonu önlenir
- Kararlı durum hatası (steady-state error) kontrol altında tutulur

5. Ölü Bant Seçiminin Mühendislik Analizi

5.1 Esik Değeri Belirleme

Ölü bant esigi (0.1 derece) şu faktörlere göre belirlenmiştir:

1. Encoder çözünürlüğü: Kullanılan encoder'ın minimum algılayabildiği açı değisimi
2. Mekanik toleranslar: Redüktör boşluğu, dişli backlash değeri
3. Sinyal gürültüsü: IMU ve encoder okumalarındaki standart sapma
4. Hedef uydu isin genişliği: Türksat uyduları için tipik isin genişliği

5.2 RSSI ve Kilit Esigi İlişkisi

`RSSI_LOCK_THRESHOLD_DB = -90.0 (config.h)`

Ölü banttan çıkıldığında hata 0.1 dereceyi aşmış demektir. Bu durumda RSSI seviyesi kontrol edilir. Eğer RSSI -90 dBm'nin altındaysa sistem alarm durumuna geçer.

5.3 Takip Hatası Alarmı

`MAX_TRACKING_ERROR_DEG = 5.0 (config.h)`

Hata 5 dereceyi aşarsa acil durum alarmı devreye girer. Bu, ölü bant esiginin 50 kati büyüklüğündedir ve sistemin tamamen hedefi kaybettiği anlamına gelir.

6. Algoritmanın Doğrulaması

6.1 Dataset Testi

Azimuth/elevasyon simülasyonunda ölü bant etkisi `simulation_output.csv` dosyasında gözlemlenebilir:

- `azimuth_deg`, `elevation_deg` (gerçek konum)
- `azimuth_motor_pwm`, `elevation_motor_pwm` (PID çıktısı)
- `boresight_error_deg` (bilesik hata)
- `locked` (kilit durumu)

Hedefe yaklaşıldığında (`boresight_error < 0.2` derece) motor PWM değeri sıfıra iner ve `locked=true` olur.

6.2 Kilit Orani Metrikleri

lock_ratio_percent: Toplam süre boyunca hedefte kalma yüzdesi

settled_lock_ratio_percent: 8. saniyeden sonra hedefte kalma yüzdesi

Bu metrikler ölü bant algoritmasının basarisi hakkında nicel veri saglar.

7. Ölü Bant Algoritmasının Sözde Kodu

```
Fonksiyon PID_ile_Ölü_Bant(hedef, ölçülen, dt):
hata = hedef - ölçülen

// Azimuth için açi sarmalama
Eger azimuth_ekseni ise:
hata = normalize_aci(hata, -180, 180)

// Ölü bant kontrolü
Eger |hata| < DEADBAND_ESIGI ise:
integral = 0
önceki_hata = 0
çikis = 0
Döndür 0 // Motor durdur

// PID hesaplama (ölü bant asildi)
integral += hata x dt
türev = (hata - önceki_hata) / dt
çikis = Kp x hata + Ki x integral + Kd x türev
çikis = clamp(çikis, min, max)

// Anti-windup
Eger çikis doyumda ve hata ayni yönde ise:
integral güncellenmez

önceki_hata = hata
Döndür çikis
```

8. Öneriler ve Gelistirme

1. Adaptif ölü bant: Rüzgar, araç titresimi gibi dis etkenlere göre ölü bant esiginin dinamik ayarlanması
2. Histerezis eklenmesi: Ölü banta giris ve çikis esiklerinin farkli olmasi (Schmitt tetikleme benzeri)
3. Ivme tabanlı ölü bant: Sadece konum hatasina degil, açisal hiz ve ivmeye de bagli esik belirleme
4. QPD entegrasyonu: Lazer takip modunda ölü bant esiginin QPD hassasiyetine göre otomatik ayarlanması

9. Sonuç

Ölü bant algoritmasi, hareketli uydu terminalinin kararli çalismasi için kritik öneme sahiptir. Projede 0.1 derece esik degeri ile uygulanan bu algoritma, motor titresimini önler, güç tüketimini azaltir ve sistem ömrünü uzatir. Integral sifirlama ve türev temizleme mekanizmalari sayesinde PID kontrolcünün genel performansi iyilestirilmistir. Bu rapor, algoritmanin projenin tüm bileşenlerindeki uygulamasini kapsamlı şekilde belgelemektedir.